

## **Experiment- 6**

|                                         |                                        |
|-----------------------------------------|----------------------------------------|
| <b>Branch:</b> MCA (AI&ML)              | <b>Semester:</b> 2                     |
| <b>Student Name:</b> Ajay Rakwal        | <b>UID:</b> 25MCI10329                 |
| <b>Subject Name:</b> Technical Training | <b>Subject Code:</b> 25CAP-652         |
| <b>Section/Group:</b> MAM-1(A)          | <b>Date of Performance:</b> 24-02-2026 |

### **Aim of the program:**

Learn how to create, query, and manage views in SQL to simplify database queries and provide a layer of abstraction for end-users.

### **Software Requirements**

- **Operating System:** Windows / Linux
- **Database Management System:** MySQL / Oracle / PostgreSQL
- **SQL Interface:** MySQL Workbench / Web Based / pgAdmin

### **Objective**

- **Data Abstraction:** To understand how to hide complex table joins and calculations behind a simple virtual table interface.
- **Enhanced Security:** To learn how to restrict user access to sensitive columns by providing views instead of direct table access.
- **Query Simplification:** To master the creation of views that pre-join multiple tables, making reporting easier for non-technical users.
- **View Management:** To understand the syntax for creating, altering, and dropping views, as well as the naming conventions required for efficient data access.

## Procedure of the Practical

- I. Open the database environment (Oracle / MS SQL Server / PostgreSQL) and connect to the required database.
- II. Create base tables such as Employees and Departments, and insert sample records for testing.
- III. Create a Simple View to filter specific records (e.g., active employees) without exposing the full table structure.
- IV. Execute SELECT queries on the simple view to verify that it retrieves filtered data correctly.
- V. Create a Join View by combining multiple tables (e.g., Employees and Departments) to simplify complex queries.
- VI. Query the join view to confirm that data from multiple tables is displayed in a consolidated format.
- VII. Create an Advanced View using aggregate functions (e.g., COUNT, AVG, SUM) and GROUP BY to generate department-level statistics.
- VIII. Test the summarization view to validate calculated outputs.
- IX. Demonstrate view management operations such as ALTER VIEW and DROP VIEW.
- X. Verify access control by granting permissions on views instead of base tables (if applicable).

## Practical / Experiment Steps

-- TABLE CREATION

```
CREATE TABLE departments (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(100)  
);  
  
CREATE TABLE employees (  
    id INT PRIMARY KEY,
```

```
name VARCHAR(100),

salary NUMERIC(10,2),

status VARCHAR(20),

department_id INT,

FOREIGN KEY (department_id)

REFERENCES departments(department_id)

);

-- INSERT SAMPLE DATA

INSERT INTO departments VALUES

(1, 'HR'),

(2, 'IT'),

(3, 'Finance');

INSERT INTO employees VALUES

(101, 'Ajay', 50000, 'Active', 2),

(102, 'Jatin', 45000, 'Inactive', 1),

(103, 'Nidhi', 60000, 'Active', 2),

(104, 'Anay', 40000, 'Active', 3);

-- Simple View

CREATE VIEW active_employees AS

SELECT id, name, department_id

FROM employees

WHERE status = 'Active';

-- Join View

CREATE VIEW employee_department_view AS
```

```
SELECT e.id, e.name, d.department_name  
  
FROM employees e  
  
JOIN departments d  
  
ON e.department_id = d.department_id;  
  
-- Summary View  
  
CREATE VIEW department_summary AS  
  
SELECT department_id,  
  
       COUNT(*) AS total_employees,  
  
       AVG(salary) AS average_salary  
  
FROM employees  
  
GROUP BY department_id;  
  
SELECT * FROM active_employees;  
  
SELECT * FROM employee_department_view;  
  
SELECT * FROM department_summary;
```

## **I/O Analysis (Input / Output)**

### **Input**

- Base table creation queries
- Sample data insertion commands
- CREATE VIEW statements (simple and complex views)
- SELECT queries on created views
- Aggregate function and JOIN queries
- ALTER VIEW and DROP VIEW commands

## Output

- Base tables created successfully
- Simple view displaying filtered employee data
- Join view showing consolidated employee-department information
- Summarization view displaying department-level statistics
- Views modified and dropped successfully
- Simplified and secure data access achieved through views

## OUTPUT:

### Table Created:

```
Data Output Messages Notifications
CREATE TABLE

Query returned successfully in 70 msec.
```

### Insert sample records:

```
Data Output Messages Notifications
INSERT 0 5

Query returned successfully in 52 msec.
```

### --Views creation

```
Data Output Messages Notifications
CREATE VIEW

Query returned successfully in 57 msec.
```

## -- Simple View

| Data Output Messages Notifications |               |                                 |                          |
|------------------------------------|---------------|---------------------------------|--------------------------|
|                                    | id<br>integer | name<br>character varying (100) | department_id<br>integer |
| 1                                  | 101           | Ajay                            | 2                        |
| 2                                  | 103           | Nidhi                           | 2                        |
| 3                                  | 104           | Anay                            | 3                        |

## -- Join View

| Data Output Messages Notifications |               |                                 |                                            |
|------------------------------------|---------------|---------------------------------|--------------------------------------------|
|                                    | id<br>integer | name<br>character varying (100) | department_name<br>character varying (100) |
| 1                                  | 101           | Ajay                            | IT                                         |
| 2                                  | 102           | Jatin                           | HR                                         |
| 3                                  | 103           | Nidhi                           | IT                                         |
| 4                                  | 104           | Anay                            | Finance                                    |

## --Summary View

| Data Output Messages Notifications |                          |                           |                           |
|------------------------------------|--------------------------|---------------------------|---------------------------|
|                                    | department_id<br>integer | total_employees<br>bigint | average_salary<br>numeric |
| 1                                  | 3                        | 1                         | 40000.000000000000        |
| 2                                  | 2                        | 2                         | 55000.000000000000        |
| 3                                  | 1                        | 1                         | 45000.000000000000        |

### **Learning Outcome:**

- Understand the concept of views and their role in data abstraction.
- Learn how to create simple and complex views using SQL syntax.
- Gain practical experience in simplifying queries using join-based and aggregate views.
- Understand how views enhance database security by restricting access to sensitive columns.
- Develop the ability to design enterprise-level reporting views for real-world applications such as payroll and management systems.